

Single-cell analysis of immune response to nanoplastic particles

0. Problem background

Plastic pollution has become one of the most widespread environmental challenges worldwide. Large plastic debris gradually degrades through physical, chemical, and biological processes, generating increasingly smaller particles that can persist in ecosystems for long periods. These particles have been detected in oceans, freshwater systems, soil, food products, and even the atmosphere, raising concerns about their potential effects on human health. As analytical methods have improved, evidence of human exposure to plastic particles has accumulated, highlighting the need to understand their biological impact at the cellular and molecular levels.

0.1 Why nanoplastic matters

Nanoplastics are generally defined as plastic particles smaller than 1 μm that originate from the degradation of larger plastic materials. Due to their small size, they may cross biological barriers more easily than larger microplastics and enter the human body through air, food, or water. Recent studies have reported the presence of plastic particles in human blood and tissues, suggesting that circulating immune cells may be exposed to these materials.

The immune system plays a central role in detecting and responding to foreign materials entering the body. *Peripheral blood mononuclear cells*, including T cells, B cells, NK cells, and monocytes, represent a heterogeneous population of immune cells that can encounter circulating nanoparticles. Changes in the composition or transcriptional programs of these cells may provide early indicators of nanoparticle-induced immune responses.

Understanding how particle size influences cellular uptake and immune activation is therefore important for assessing potential health risks associated with long-term exposure.

0.2 PSNPs used in this study

Material: carboxylated polystyrene nanoparticles (PSNPs).

- 40 nm particles (Sample 1): very small; high surface-area-to-volume ratio.
- 200 nm particles (Sample 2): larger; different uptake and cellular handling expected.
- 40 + 200 nm mixture (Sample 3): tests combined exposure
- Control (Sample 4): same donor, no nanoparticle exposure.

0.3 Experimental limitations (state in every report)

- Single donor: results may not generalize to all people.
- In vitro exposure design (cells exposed outside the body): simplifies real-world complexity.
- RNA only: transcriptome reflects gene activity, not direct protein levels or full cell function.
- No spatial information: we see cell types and genes, not tissue anatomy.

0.4 Hypothesis and project aims

Hypothesis: nanoparticle size alters immune cell composition and gene expression programs in human PBMC.

Aims:

- Identify which immune cell types change in proportion after exposure.
- Find genes and pathways altered by 40 nm, 200 nm, and mixture vs control.
- Separate size-specific, shared, and mixture-only transcriptional responses.

1. Biological context

Nanoplastics are very small plastic particles (often in the nanometer range). When they enter the blood, they can interact with **immune cells** in peripheral blood. This project studies whether **particle size** changes that response:

Sample	Treatment
Sample 1	40 nm carboxylated polystyrene nanoparticles (PSNPs)
Sample 2	200 nm PSNPs
Sample 3	Mixture of 40 nm + 200 nm
Sample 4	<i>Control</i> — no nanoparticles

All four samples come from **one donor**. Cells are **PBMC** (*peripheral blood mononuclear cells*): mainly T cells, B cells, NK cells, monocytes, and some dendritic cells.

We use **single-cell RNA sequencing (scRNA-seq)**. Instead of averaging RNA across millions of mixed cells (bulk RNA-seq), we measure RNA **in each cell separately**. That helps see:

- Which cell types respond to nanoplastics?
- Which genes change in each type?
- Are effects specific to 40 nm, 200 nm, or only seen in the mixture?

The scientific question: **How does nanoparticle size reshape the immune transcriptome at single-cell resolution?**

2. Data

Zenodo (<https://zenodo.org>) is a free, open repository for research data, software, and publications. Researchers use it as cloud storage for datasets:

- Each upload gets a permanent DOI (e.g. 10.5281/zenodo.15866724)
- Data are public (this dataset uses an open license, CC BY 4.0)
- Others can download the same files and reproduce the analysis

Our main dataset:

<https://zenodo.org/records/15866724>

2.1 File *download_data.py*:

1. Load config/config.yaml

→ zenodo_record_id: "15866724"

→ azimuth.record_id: "4546839" (optional)

2. **GET** <https://zenodo.org/api/records/15866724>

→ JSON with file list, sizes, download links

3. Save zenodo_record.json in data/raw/

(metadata: title, description, DOI, checksums — not expression data)

4. For each file ending in .h5ad, .csv, .rds, or .annoy:

→ download to data/raw/<filename>

→ if file already exists → [SKIP]

5. Repeat steps 2–4 for record 4546839 (Azimuth reference)

→ zenodo_record_4546839.json + ref.Rds + idx.annoy

Stream download: files are written in 8 KB chunks so a ~200 MB .h5ad is not loaded entirely into RAM at once.

2.2 What ends up in data/raw/ after download:

A) Main dataset (Zenodo record 15866724) — ~805 MB

1. Four .h5ad files (core scRNA-seq data)

File on disk	Experimental condition	Size (approx.)	Contents
filtered_feature_bc_matrix.h5ad	Sample 1 — 40 nm PSNPs	192 MB	AnnData: expression matrix + cells
filtered_feature_bc_matrix_Sample2.h5ad	Sample 2 — 200 nm	260 MB	same
filtered_feature_bc_matrix_Sample3.h5ad	Sample 3 — 40+200 nm mix	159 MB	same
filtered_feature_bc_matrix_Sample4.h5ad	Sample 4 — control	190 MB	same

Inside each file (AnnData / Scanpy):

- `adata.X` — UMI/read counts per cell and gene (already filtered in the published pipeline)
- `adata.obs` — one row per cell (barcode, QC metrics if present)
- `adata.var` — one row per gene (gene symbols)

The name `filtered_feature_bc_matrix` comes from the 10x Genomics format: a feature–barcode matrix that has already been filtered (low-quality cells removed before publication).

Mapping in your `config.yaml`:

File	condition in pipeline
<code>filtered_feature_bc_matrix</code>	<code>PSNP_40nm</code>
<code>filtered_feature_bc_matrix_Sample2</code>	<code>PSNP_200nm</code>
<code>filtered_feature_bc_matrix_Sample3</code>	<code>PSNP_mix_40_200</code>
<code>filtered_feature_bc_matrix_Sample4</code>	<code>control</code>

2. Four *_CoDi_KLD.csv files (reference cell-type labels)

File	Sample	Size (approx.)
filtered_feature_bc_matrix_CoDi_KLD.csv	Sample 1 (40 nm)	0.6 MB
filtered_feature_bc_matrix_Sample2_CoDi_KLD.csv	Sample 2	0.9 MB
filtered_feature_bc_matrix_Sample3_CoDi_KLD.csv	Sample 3	0.4 MB
filtered_feature_bc_matrix_Sample4_CoDi_KLD.csv	Sample 4	0.5 MB

What **CoDi** is: *an external model that predicts a cell type for each cell* (e.g. CD4+ T cell, B cell, CD14+ monocyte).

Example columns:

Column	Meaning
cell_id	Cell barcode ID
CoDi	Predicted cell type
CoDi_confidence	Confidence score (0–1)
CoDi_dist, CoDi_contrastive	Alternative prediction modes from the same framework

In our pipeline, CoDi is used to compare with marker-based labels (load_codi_annotations), not as the only source of truth.

3. zenodo_record.json (a few KB)

- A copy of the Zenodo API response
- Title, description, DOI, file list, checksums
- Useful for citations in your report; *not used in expression analysis*

B) Azimuth reference (Zenodo record 4546839) — ~82 MB

Azimuth is a tool from the Satija Lab (same group behind Seurat) *for automatic cell-type annotation in single-cell data*. Instead of you picking marker genes by hand, it:

1. Takes your cells (query dataset)
2. Compares them to a large reference atlas of well-characterized PBMC cells
3. Assigns each cell a predicted cell type at three levels of detail

If config.yaml has azimuth.record_id: "4546839", the script also downloads:

File	Size (approx.)	Purpose
ref.Rds	51 MB	PBMC reference atlas (R / Seurat / Azimuth)
idx.annoy	31 MB	Index for fast mapping of cells to the reference
zenodo_record_4546839.json	small	Metadata for that upload

Used by scripts/azimuth_annotation.R to map cells onto the Satija lab PBMC reference.

We have three ways to label cells:

Method	Where	How it works
Marker-based	run_pipeline.py	Rules: high CD3D → T cell, high LYZ → monocyte, etc.
CoDi	CSV from main dataset	Pre-computed labels from authors' external model
Azimuth	Separate R script	Model that maps cells to Satija PBMC reference atlas

Overview

Data comes from Zenodo, an open repository (DOI: 10.5281/zenodo.15866724). The script download_data.py fetches four AnnData (.h5ad) files with scRNA-seq measurements of PBMC cells under four conditions (40 nm, 200 nm, mixture, control), plus CoDi CSV files for reference cell-type labels and optionally the Azimuth reference (ref.Rds, idx.annoy). Everything is stored in data/raw/. The pipeline then loads the .h5ad files and runs QC, integration, clustering, differential expression, and pathway analysis.

3. How genes and cells are represented and how they relate

3.1 The basic idea: a matrix

scRNA-seq data is stored as a **count matrix**:

	Gene A	Gene B	Gene C	...	Gene ~20,000
Cell 1	0	5	120	...	3
Cell 2	12	0	8	...	45
...	...	...	...	...	...

- **Each row** = one **cell** (one biological unit).
- **Each column** = one **gene** (one feature you measure).
- **Each value** = how many **UMIs / reads** were detected for that gene in that cell (expression strength). Approximate number of captured mRNA molecules from the gene were detected after sequencing and deduplication, reflecting how highly that gene is transcribed in that cell relative to other genes and other cells.

So the relationship is: One cell has many genes measured; one gene is measured in many cells. In code (Scanpy / AnnData), this is usually stored as `adata.X`: cells × genes (rows = cells, columns = genes in AnnData convention).

3.2 What is a “cell” in the data?

Each captured cell gets:

1. **Barcode** — unique ID, e.g. AAACCCAAGACGCATG-1
 - In `adata.obs_names` (row index of the matrix).
2. **Metadata (obs)** — information about the cell, not expression:
 - `sample_id` — which file / batch (Sample2, Sample4, ...)
 - `condition` — PSNP_40nm, PSNP_200nm, PSNP_mix_40_200, control
 - `cell_type_marker` — T cell, B cell, monocyte, ... (after annotation)
 - `cluster` — Leiden cluster number
 - QC: `n_genes_by_counts`, `total_counts`, `pct_counts_mt`

Cell = identity (barcode) + metadata (labels) + expression profile (thousands of numbers)

3.3 What is a “gene” in the data?

Each gene has:

1. **Gene symbol** — e.g. CD3D, LYZ, ISG15
 - In *adata.var_names* (column index).
2. **Optional gene metadata (var)** — e.g. whether it’s highly variable (*highly_variable*), mitochondrial (*mt*).

A **gene** is a column: its values across all cells form an **expression vector**. That is a list of expression values (UMI counts), one number per cell, showing how strongly that gene is detected across the entire dataset.

For example, the gene *LYZ* might have values [2, 120, 3, ...] across thousands of cells—high in monocytes, low in T cells. Differential expression analysis compares these vectors between conditions (e.g. control vs 40 nm exposure) to see whether a gene is up- or down-regulated.

3.4 Relationship between cells and genes (intuition)

GENES (features)		
CD3D LYZ MS4A1 ...		
Cell 1	high low low	→ T cell profile
(T cell)		
Cell 2	low high low	→ Monocyte profile
(mono)		
Cell 3	low low high	→ B cell profile
(B cell)		

- **Similar cells** → similar rows

Each row is one cell's expression across all genes. Cells of the same type (e.g. two T cells) tend to have similar values in the same columns, so their rows look alike. That's why they cluster together on a UMAP.

- **Marker genes** → columns high only in certain cell types

A marker gene is strongly expressed in one cell type and weak elsewhere. That column is high in rows of that type (e.g. LYZ high in monocytes) and low in others—used to label cell types.

- **DE analysis** → compare gene columns within one cell type (control vs 40 nm)

You first keep only one cell type (same rows). Then, for each gene (column), you test whether expression differs between control and 40 nm cells. That finds genes up- or down-regulated by treatment in that type, without mixing T cells, B cells, etc.

3.5 AnnData: how Python organizes this

AnnData = annotated matrix. Main pieces:

Object part	Meaning	Analogy
<code>adata.X</code>	Main expression (often normalized/log)	The big number table
<code>adata.layers["counts"]</code>	Raw counts before normalization	Backup of original counts
<code>adata.obs</code>	One row per cell — metadata	Cell “ID card”
<code>adata.var</code>	One row per gene — metadata	Gene “ID card”
<code>adata.obsm["X_umap"]</code>	2D coordinates per cell	Map position for plotting
<code>adata.obs["cell_type_marker"]</code>	Label per cell	Cell type name

Relationship rule:

- adata.obs has **same number of rows** as adata.X.
- adata.var has **same number of rows** as adata.X has **columns**.
- Row i of X = expression of all genes in cell i .

3.6 From raw counts to what we analyze

Pipeline steps change the numbers but **not** the cell–gene structure:

1. **Raw counts** (layers["counts"])
 - Integer counts from sequencing.
2. **Normalization** (normalize_total + log1p)
 - Cells are sequenced to different depths; you scale so comparisons are fair.
 - Log makes statistics more stable.
3. **HVG selection**
 - Keep ~3000 genes that vary most across cells; drop the rest from downstream PCA/clustering.
4. **PCA / Combat / UMAP**
 - Compress and correct **cells** using **gene** patterns.
 - UMAP: each cell $\rightarrow$ 2 points (x, y) for plotting.
5. **Annotation**
 - Each cell gets a type by checking which **marker genes** are highest.
6. **DE**
 - Within one cell type, compare **each gene's** distribution: exposure vs control.

So genes and cells stay linked: you never analyze genes without knowing which cells they came from, and you ne

3.7 How your four conditions fit in

All cells share the **same genes** (columns). They differ by **metadata**:

Same matrix structure

- |— subset: condition == "control" → control cells
- |— subset: condition == "PSNP_40nm" → 40 nm cells
- |— subset: condition == "PSNP_200nm" → 200 nm cells
- |— subset: condition == "PSNP_mix..." → mix cells

Composition analysis — count cells per type per condition (rows grouped by labels).

DE — for one cell type, compare gene columns between two conditions.

Size-specific — compare *sets of genes* that changed in 40 nm vs 200 nm vs mix.

3.8 CoDi and markers — two ways to label cells

Method	Idea
Marker-based (code)	If LYZ, S100A8 are high → monocyte. Rule-based from known biology.
CoDi (dataset)	External model's prediction per barcode. Used to validate your labels.
Azimuth (R, optional)	Map cells to a PBMC reference atlas. Finer labels (e.g. CD4 T vs CD8 T).

All methods assign **one label per cell**; genes are still the evidence underneath.

3.9 One concrete example

Question: Do 40 nm particles activate monocytes?

1. Select cells where cell_type_marker == Monocyte_CD14.
2. Split into control vs PSNP_40nm.
3. For each gene (column), test if expression differs (Wilcoxon → DE table).
4. Significant genes → pathway enrichment (e.g. inflammation, interferon).

Cells = which population you compare.

Genes = what actually changed inside that population.

4. Project implementation

Assignment task	Implementation in this project
QC & preprocessing	read_and_qc_sample(); thresholds in config/config.yaml
Integration & clustering	merge_and_integrate(); Combat, PCA, UMAP, Leiden
Cell type annotation	marker_based_annotation(); CoDi CSV; optional Azimuth (R)
Composition analysis	composition_analysis(); barplot + CSV
Differential expression	differential_expression_by_celltype(); Wilcoxon vs control
Pathway enrichment	pathway_enrichment(); GO, KEGG, Reactome via gseapy/Enrichr
Size-specific effects	size_specific_effects(); unique/shared/mix-only gene sets
3–5 extra analyses	additional_insights(): cell cycle, IFN, antigen presentation, pseudobulk, CoDi agreement

4.1 run_pipeline.py — step-by-step

Step	Function	What it does	Output
1	read_and_qc_sample (×4)	Load each .h5ad; filter bad cells; add condition	4 cleaned sample objects
2	merge_and_integrate	Merge samples; normalize; HVG; PCA; Combat; UMAP; Leiden	Single integrated AnnData
3	marker_based_annotation	Score marker genes; assign cell_type_marker	obs column cell_type_marker
4	load_codi_annotations	Map CoDi CSV labels to cells	obs column cell_type_codi
5	save_core_figures	Export UMAP and composition plots	results/figures/*.png

6	composition_analysis	Cell type fractions per condition	cell_composition_by_condition.csv
7	differential_expression_by_celltype	Wilcoxon: exposure vs control per cell type	differential_expression_all.csv
8	pathway_enrichment	GO/KEGG/Reactome on significant DE genes	pathway_enrichment_all.csv
9	size_specific_effects	Classify DE genes by 40/200/mix	size_specific_effects_summary.csv
10	additional_insights	IFN, cell cycle, antigen presentation, pseudobulk, agreement	multiple CSV files
11	adata.write	Save final object	data/processed/integrated_annotated.h5ad

4.2 read_and_qc_sample

Loads one sample's .h5ad, annotates it, computes QC metrics, filters low-quality cells, and returns a cleaned AnnData object.

```
sc.read_h5ad(file_path)
```

- Loads the .h5ad file into an AnnData object (X, obs, var, ...).

```
n_before = adata.n_obs
```

- Stores the cell count before filtering.

```
adata.obs["sample_id"] = ... / ["condition"] = ...
```

- Adds sample and treatment labels to each cell's metadata.

```
adata.var["mt"] = adata.var_names.str.upper().str.startswith("MT-")
```

- Flags mitochondrial genes (names starting with MT-).

```
sc.pp.calculate_qc_metrics(adata, qc_vars=["mt"], inplace=True)
```

- Adds per-cell QC columns in obs, e.g. n_genes_by_counts, total_counts, pct_counts_mt...
- For each cell, it derives standard QC metrics from adata.X and stores them in obs.
- *pct_counts_mt* is not inferred from gene names alone. We need to:
 - Mark MT genes: **adata.var["mt"] = ...**
 - Pass: **qc_vars=["mt"]**
 - Then the function also adds MT-related columns, including pct_counts_mt (fraction of counts from MT genes). Without this, we still get the basic metrics, but not mitochondrial percentage.

Boolean indexing + .copy()

- Keeps only cells passing all thresholds from qc_cfg: min/max genes, min UMI counts, max MT%.
- *copy()* makes a new object, independent from the original

adata.X.astype(np.float32)

- Casts the expression matrix to float32 to save memory.

n_after = adata.n_obs

- Stores the cell count after filtering.

*pct_kept = 100.0 * n_after / n_before if n_before else 0*

- Calculates the percentage of cells kept, for logging.

log.log(...)

- Logs cells before → after QC and % retained.

QC thresholds (from config/config.yaml): min_genes 200, max_genes 7000, min_counts 500, max_mt_percent 15.

Pipeline usage: called once per sample file in the main loop (read_and_qc_sample(f, sample_id, meta["condition"], cfg["qc"], log)), then samples are concatenated downstream.

Returns the filtered, annotated sample.

4.3 merge_and_integrate

Takes the already concatenated AnnData (4 QC-filtered samples merged in main() via `sc.concat`) and runs normalization, HVG selection, scaling, PCA, batch correction, neighbor graph, UMAP, and Leiden clustering.

Note: Sample merging happens before this function (lines 658–665). `merge_and_integrate` handles preprocessing + integration, not `sc.concat`.

Inputs: merged `adata`, `cfg["preprocessing"]`, `log`.

Output: processed AnnData with PCA, integration representation, UMAP, and `obs["cluster"]`.

Config defaults: 3000 HVGs, 30 PCs, neighbors_k 15, Leiden resolution 0.5, integration_method: "combat".

`adata.obs_names_make_unique()`

- Ensures unique cell barcodes across samples (safety after concat).

`adata.layers["counts"] = adata.X.copy()`

- Saves raw (post-QC) counts before normalization so DE can use original counts later.

`sc.pp.normalize_total(adata, target_sum=1e4)`

- Cells have different total UMI counts (different sequencing depth). A cell with 20,000 UMIs looks more “highly expressed” than one with 2,000 UMIs, even when the difference is often technical, not biological.
- For each cell, this function scales all gene values so the **total counts per cell = 10,000** (proportional scaling).
- Makes sequencing depth comparable across cells.

`sc.pp.log1p(adata)`

- The log reduces the impact of extreme values and makes distributions easier to work with.
- After normalization you have relative expression; after log1p you have a log-normalized matrix suited for HVG selection, scaling, and PCA.

`sc.pp.highly_variable_genes(..., flavor="seurat", batch_key="sample_id")`

- Marks ~3000 highly variable genes – **HVGs** - per sample-aware Seurat-style selection; adds `var["highly_variable"]`.
- HVGs are genes whose expression **varies most across cells** (biological signal), not genes that are flat or mostly noise.
 - Genes that vary most — main signal for analysis.
- Out of ~20,000 genes, most are nearly constant. For clustering/UMAP, ~3,000 HVGs (your config: `n_hvgs: 3000`) are usually enough.

```
adata = adata[:, adata.var["highly_variable"]].copy()
```

- Subsets matrix to HVGs only (smaller, faster downstream steps).

```
sc.pp.scale(adata, max_value=10)
```

- Z-score per gene; clips scaled values at ± 10 .
- Genes with large absolute counts would otherwise dominate PCA just because they have large variance on the original scale.

```
sc.tl.pca(adata, n_comps=n_pcs, svd_solver="arpack")
```

- **PCA = Principal Component Analysis** — linear dimensionality reduction.
 - Compresses many genes into a few components.
- PCA finds directions in gene space where the data vary the most (PC1 = strongest shared signal, PC2 = next, etc.). Each cell becomes a 30-number vector instead of 3,000.
- Input: thousands of genes (HVGs) × thousands of cells.
- Output: e.g. 30 components (`n_pcs: 30`) per cell, stored in `adata.obsm["X_pca"]`.

```
sc.external.pp.harmony_integrate(adata, key="sample_id")
```

- If `integration_method == "harmony"` and Harmony works: corrects batch across samples in PCA space → `obsm["X_pca_harmony"]`.
- Harmony/Combat: removes batch differences between samples.

```
sc.pp.combat(adata, key="sample_id")
```

- Default / fallback: ComBat batch correction on PCA by `sample_id`; uses `X_pca` as representation.

```
sc.pp.neighbors(adata, use_rep=rep_key, n_neighbors=k)
```

- Builds **kNN graph** ($k=15$) using corrected PCA (`X_pca` or `X_pca_harmony`).
- For each cell, find the k nearest neighbors.

- **How it “uses PCA”:** Distances are not computed on 3,000 genes; they’re computed on 30-dimensional vectors (usually Euclidean distance). Two cells are “similar” if they’re close in PCA space.
- Output: A neighborhood graph (who is whose neighbor) plus matrices for UMAP/Leiden, typically in `adata.obsp`.
- Intuition: “Who looks like whom?” — local structure of the data.

`sc.tl.umap(adata)`

- **2D UMAP - Uniform Manifold Approximation and Projection** (a 2D layout of thousands of cells) - embedding from the neighbor graph → `obsm["X_umap"]`.
- How (simplified):
 - Start from the KNN graph (local relationships).
 - Find a 2D layout where close neighbors in the graph stay close on the plot, and distant points are pushed apart.
 - Result: `adata.obsm["X_umap"]` — two coordinates per cell for plotting.
- Used for visualization

`sc.tl.leiden(adata, resolution=res, key_added="cluster")`

- Community detection clustering → `obs["cluster"]`
- **Community detection** = Finds groups of nodes that are densely connected inside and weakly connected between groups — “communities” in the graph.
- Splits cells into clusters based on graph density.

Pipeline usage: called once after all samples are concatenated: `adata = merge_and_integrate(adata, cfg, log)`.

Returns integrated, embedded, clustered object.

4.4 marker_based_annotation

Labels each cell with **the cell type** whose marker genes (from config/config.yaml → `cfg["markers"]`) have the **highest mean expression** in that cell, writing the result to `cell_type_marker`.

This is a simple rule-based method, not a machine-learning classifier. It does not modify `adata.X`; it only adds a column to `obs` and `log` counts.

For each cell and each cell type:

- Take that type's marker genes that exist in the dataset.
- Compute the mean expression of those markers in that cell.
- Assign the cell type with the highest mean score ("winner takes all").

So: "Which marker program looks strongest in this cell?"

```
gene_set = set(adata.var_names)
```

- All gene names currently in `adata` (after HVG subset — see note below)

```
valid = [g for g in markers if g in gene_set]
```

- Keeps only markers present in the matrix.
- If `valid` is empty - score = -inf for that type (it can never win).

```
adata[:, valid].X.mean(axis=1)
```

- Per cell: average expression of valid markers → one score per type.

```
pd.DataFrame(score_matrix, ...)
```

- Table: rows = cells, columns = cell types, values = scores.
- The `DataFrame` organizes per-cell, per-type marker scores so `idxmax` can assign each cell to the type with the highest mean marker expression in a clean, vectorized way.

```
scores_df.idxmax(axis=1)
```

- For each cell, pick the column (cell type) with the max score.

```
adata.obs["cell_type_marker"] = ...
```

- Save label per cell.

```
value_counts() + log.log
```

- Log how many cells per type.

merge_and_integrate subsets to HVGs only (~3000 genes). Annotation runs on that matrix, so a marker counts only if it survived HVG selection and is still in `adata.var_names`. Missing markers are skipped; if all markers for a type are missing, that type gets -inf and cannot be assigned.

Usage later:

- UMAP colored by `cell_type_marker`
- Composition plots by condition × cell type
- DE analysis **within** a chosen type (e.g. monocytes only)
- Agreement check vs `cell_type_codi_norm` (CoDi labels)

4.5 load_codi_annotations

Load **external CoDi labels** from CSV files bundled with the Zenodo dataset and attach them to cells in `adata`. This does not run a new annotation algorithm — only import + barcode matching. It supports comparison/validation of your marker labels (`cell_type_marker`).

Does not modify `adata.X`. Only adds columns to `adata.obs`.

CoDi = *external predictions shipped with the dataset* (`*_CoDi_KLD.csv` in `data/raw/`). Each cell already has a predicted type (e.g. "CD4+ T cell", "CD14+ monocyte").

The assignment says CoDi files are a reference for comparing your results — that is what this function implements.

```
glob("*_CoDi_KLD.csv")
```

- Finds all CoDi CSV files in `data/raw/` (usually one per sample).

```
pd.read_csv(f)
```

- Loads each file.

Keep cell_id + CoDi

- Keeps only cell ID and CoDi label because only they are needed
- Renames CoDi → cell_type_codi.

If no files

- Sets cell_type_codi = "NA" for all cells, logs a warning, exits.

pd.concat + drop_duplicates

- Merges all samples; duplicate cell_id rows are dropped.

cell_id_clean (regex -\d+\$)

- Makes cell barcodes match between adata and the CoDi CSV by removing a -number suffix at the end.
- Strips trailing suffixes like -1, -2 from barcodes (Scanpy often adds these after concat).

map(...) onto adata.obs

- This step assigns each cell in adata its CoDi label by matching cleaned barcodes to the combined CoDi table.
- Looks up the CoDi label for each cell using the cleaned barcode.

.fillna("NA")

- Cells with no match → "NA".

Log % mapped

- Reports what fraction of cells received a CoDi label.

codi_to_marker dictionary

- Maps CoDi names to the same short names used in config marker types.

cell_type_codi_norm

- Normalized CoDi label for fair comparison with cell_type_marker.
- The problem is two naming systems:
 - After mapping, you have **cell_type_codi** with CoDi strings, e.g.:
 - "CD4+ T cell"
 - "CD14+ monocyte"
 - Your marker step stores **cell_type_marker** with config names, e.g.:

- "CD4_T"
- "Monocyte_CD14"
- Same biology, different spelling. You can't meaningfully ask "do they agree?" if you compare "CD4+ T cell" to "CD4_T" — they'll always look different even when both mean CD4 T cells.
- Later the pipeline checks agreement:


```
cell_type_codi_norm == cell_type_marker
```
- That only works if both columns use the same label set (CD4_T, B_cell, ...).

4.6 composition_analysis

Compare **how many cells of each type** you have in each of the four exposure conditions (40 nm, 200 nm, mixture, control). Answers: "Did nanoplastic exposure change the proportion of T cells, monocytes, B cells, etc.?"

Inputs: merged, annotated adata (needs obs["condition"] and obs["cell_type_marker"]), output paths, logger.

Output: DataFrame comp, CSV table, barplot PNG.

Called in: STEP 5 — after annotation, before differential expression.

Does not modify adata.X or cell labels — only counts and plots what is already in obs.

```
groupby(["condition", "cell_type_marker"]).size()
```

- For each **condition × cell type** pair, counts how many cells fall in that group.

```
.reset_index(name="n_cells")
```

- Turns the group counts into a table with columns condition, cell_type_marker, n_cells.

```
.sort_values(...)
```

- Sorts by condition, then by n_cells descending (largest types first within each condition).

```
comp["fraction"] = n_cells / sum per condition
```

- Converts counts to fractions (0–1): each type's share of all cells in that condition.

`comp.to_csv(...)`

- Saves results/tables/cell_composition_by_condition.csv.

`sns.barplot(..., x="condition", y="fraction", hue="cell_type_marker")`

- Bar chart: x = condition, bar height = fraction, colour = cell type.
- The barplot x-axis = four conditions (from each sample).

`plt.savefig(...)`

- Saves results/figures/composition_barplot.png.

Returns the table for use elsewhere if needed.

4.7 differential_expression_by_celltype

For each major cell type, **find genes whose expression differs between each nanoplastic exposure and control**. Answers: “Within monocytes (or T cells, B cells, etc.), which genes go up or down after 40 nm, 200 nm, or mixture exposure?”

Inputs: merged, annotated adata (needs `obs["condition"]` and `obs["cell_type_marker"]`), `cfg["de"]`, output paths, logger.

Output: DataFrame `de_all`, CSV table results/tables/differential_expression_all.csv.

Uses `adata.X` (processed expression after normalization/log/HVG in the pipeline). Does not change cell labels — runs statistical tests and saves gene-level results.

Config: `min_cells_per_group=20`, `pval_adj_threshold=0.05`, `logfc_threshold=0.25`.

`conditions = ["PSNP_40nm", "PSNP_200nm", "PSNP_mix_40_200"]`

- Three exposed groups; control is the reference in each comparison (not listed here).

for ctype in sorted(adata.obs["cell_type_marker"].unique()):

- Loops over each annotated cell type (T, B, monocyte, etc.) separately.

`ad_ct = adata[adata.obs["cell_type_marker"] == ctype].copy()`

- Subsets adata to one cell type only so DE compares like-with-like cells.

```
if ad_ct.n_obs < cfg["de"]["min_cells_per_group"]: continue
```

- Skips the entire cell type if it has fewer than 20 cells total.

```
for cond in conditions:
```

- For that cell type, runs three comparisons: 40 nm vs control, 200 nm vs control, mixture vs control.

```
n_test = (ad_ct.obs["condition"] == cond).sum()
```

```
n_ctrl = (ad_ct.obs["condition"] == "control").sum()
```

```
if min(n_test, n_ctrl) < cfg["de"]["min_cells_per_group"]: continue
```

- Counts cells in exposed vs control arms; skips comparison if either group has fewer than 20 cells.

```
ad_ct.obs["de_group"] = np.where(ad_ct.obs["condition"] == cond, cond, "control")
```

- Temporary label: cells in the current exposure get the condition name; all other cells in this subset are labeled "control".

```
ad_sub = ad_ct[ad_ct.obs["de_group"].isin([cond, "control"])].copy()
```

- Keeps only the current exposure and control cells; drops the other two exposure conditions for this test.

```
sc.tl.rank_genes_groups(ad_sub, groupby="de_group", groups=[cond], reference="control",  
method="wilcoxon")
```

- Scanpy differential expression: for each gene, Wilcoxon rank-sum test — exposed vs control within this cell type.

```
de_df = sc.get.rank_genes_groups_df(ad_sub, group=cond)
```

- Extracts results into a table: gene name, log fold change, p-value, adjusted p-value, score.

```
de_df["cell_type"] = ctype
```

```
de_df["comparison"] = f"{cond}_vs_control"
```

- Tags each row with cell type and comparison name (e.g. Monocyte_CD14, PSNP_40nm_vs_control).

```
all_de.append(de_df) + pd.concat(all_de, ignore_index=True)
```


- Stacks all cell-type × condition results into one long table.

de_all.to_csv(... differential_expression_all.csv ...)

- Saves full DE results to results/tables/differential_expression_all.csv.

Log significant hit count

(pvals_adj < 0.05) & (|logfoldchanges| > 0.25)

- Reports how many gene rows pass significance thresholds (used downstream; full table is always saved).

Returns the table for pathway enrichment and size-specific effects.

How to read results: Positive logFC → higher in exposed; negative logFC → higher in control. Each row = one gene in one cell type for one exposure vs control comparison.

4.8 pathway_enrichment

After DE finds which genes change, this asks which biological pathways/processes those genes point to (GO, KEGG, Reactome). Answers: “In monocytes exposed to 40 nm PSNP, are upregulated genes enriched for inflammation, interferon, antigen presentation, etc.?”

Inputs: de_all from differential expression, cfg[“de”] thresholds, output paths, logger.

Output: DataFrame enr_all, CSV table results/tables/pathway_enrichment_all.csv.

Does not modify adata — reads the DE table and queries Enrichr via gseapy (gp.enrichr).

Note: Requires network access to Enrichr.

Uses same significance cutoffs as DE: pval_adj_threshold=0.05, logfc_threshold=0.25 (upregulated genes only).

Differential expression gives you many gene names — often hundreds. Reading each gene is slow and hard to summarize.

Pathway enrichment **groups genes into themes** already defined by databases:

- **GO** (Gene Ontology) — e.g. “inflammatory response”, “cell migration”

- **KEGG** — e.g. “TNF signaling pathway”, “Cytokine–cytokine receptor interaction”
- **Reactome** — detailed reaction/pathway maps

So instead of “IL1B, TNF, CXCL8, ...” you can say “*inflammatory / immune activation pathways are enriched.*”

How it works (intuition)

1. You have a gene list of interest — in your project: significantly upregulated genes after PSNP exposure in a given cell type.
2. Databases contain **pathways** = sets of genes known to work together in a process.
3. Enrichment tests: Does your list contain unusually many genes from pathway X?
 - Example: 15 of your 80 DE genes are in “TNF signaling”, but random lists of 80 genes would usually only hit 3 → enriched (likely biology, not coincidence).

Enriched = gene list overlaps that pathway more than expected

Tools like Enrichr (used in your pipeline) compute overlap and a p-value for each pathway.

Goal: *interpret how immune cells respond to particle size, not just which single genes moved*

We use:

- Python library: import **gseapy** as gp
- Backend service – enrichment engine: Enrichr (via gp.enrichr)
- Databases (gene set libraries, pathwat collections): GO_Biological_Process_2023, KEGG_2021_Human, Reactome_2022

if de_all.empty: return

- Skips pathway enrichment if DE produced no results.

for (ctype, comp), df in de_all.groupby(["cell_type", "comparison"]):

- Loops over each *cell type* × *comparison chunk* (e.g. Monocyte_CD14 + PSNP_40nm_vs_control).

Filter significant upregulated genes:

(pvals_adj < 0.05) & (logfoldchanges > 0.25)

- Keeps genes that are statistically significant and higher in exposure than control (not downregulated).

if len(sig) < 10: continue

- Skips Enrichr if fewer than 10 significant up genes — enrichment is unreliable with very small lists.

gene_sets = ["GO_Biological_Process_2023", "KEGG_2021_Human", "Reactome_2022"]

- Three pathway libraries matching the assignment: GO, KEGG, Reactome.

gp.enrichr(gene_list=list(sig), gene_sets=gset, organism="Human", outdir=None)

- Sends gene list to Enrichr; tests whether those genes overlap pathway gene sets more than expected by chance.

tmp["cell_type"] = ctype / tmp["comparison"] = comp / tmp["gene_set"] = gset

- Tags each enrichment result row with cell type, comparison, and database name.

pd.concat(enr_frames) + enr_all.to_csv(... pathway_enrichment_all.csv ...)

- Merges all enrichment runs into one table and saves to results/tables/pathway_enrichment_all.csv.

Returns the table for logging / optional downstream use.

How to read results: Each row = one enriched pathway/term for one cell type and one exposure vs control. Look at Term (pathway name), Adjusted P-value (significance), and Genes (which DE genes drove the enrichment). Use in report/slides to interpret biological mechanisms behind nanoplastic exposure (e.g. inflammatory response in monocytes at 40 nm).

4.9 size_specific_effects

Compares which DE **genes respond to which particle size** — unique to 40 nm, unique to 200 nm, shared between sizes, shared by all three exposures, or only seen in the mixture. Answers: “Does 40 nm trigger different genes than 200 nm? Do mix-only effects appear that neither size alone shows?”

Inputs: `de_all` from differential expression, `cfg["de"]` thresholds, output paths, logger.
Output: DataFrame out, CSV table `results/tables/size_specific_effects_summary.csv`.

Does not modify `adata` — only classifies genes already in the DE table.

Uses significance cutoffs: $pval_adj < 0.05$ and $|\log FC| > 0.25$ (both up- and down-regulated genes, unlike pathway enrichment).

if de_all.empty: return

- Skips if DE produced no results.

Filter significant DE genes:

(pvals_adj < 0.05) & (|logfoldchanges| > 0.25)

- Keeps all significantly changed genes (up or down); keeps columns `cell_type`, `comparison`, `names`.

for ctype in sig["cell_type"].unique():

- Analyzes each cell type separately (monocytes, T cells, etc.).

s40 = set(... PSNP_40nm_vs_control ...) s200 = set(... PSNP_200nm_vs_control ...) smix = set(... PSNP_mix_40_200_vs_control ...)

- Three gene sets per cell type: significant genes for 40 nm, 200 nm, and mixture vs control.

unique_40nm = len(s40 - s200 - smix)

- Genes significant only for 40 nm — not for 200 nm or mixture → 40 nm-specific response.

unique_200nm = len(s200 - s40 - smmix)

- Genes significant only for 200 nm → 200 nm-specific response.

shared_40_200 = len((s40 & s200) - smix)

- Genes significant for both 40 nm and 200 nm, but not in mixture → shared size-independent PSNP effect (both single sizes, not mix).

shared_all_three = len(s40 & s200 & smix)

- Genes significant in all three exposures vs control → general nanoplastic response (any PSNP condition).

mix_only_emergent = len(smix - s40 - s200)

- Genes significant only in mixture, not in 40 nm or 200 nm alone → emergent mix-only effect (possible synergy when both sizes combined).

out.to_csv(... size_specific_effects_summary.csv ...)

- Saves one row per cell type × effect class with gene counts.

Returns summary table for report/slides.

How to read results: Table columns: cell_type, effect_class, n_genes. High unique_40nm in monocytes → many genes respond specifically to small particles; high mix_only_emergent → mixture triggers responses neither size alone shows. Use with pathway enrichment and biological interpretation in slides (assignment: size-specific effects).

4.10 main

When you run *python scripts/run_pipeline.py*, execution starts at *main()*. First it loads settings with *load_config()* from *config/config.yaml* and prepares output folders with *setup_paths(cfg)*. It then calls *describe_raw_data()* to summarize the four input .h5ad files before analysis begins.

The pipeline processes each sample in a loop. For every sample it calls *read_and_qc_sample()* to load the file, filter low-quality cells, and attach metadata (sample_id, condition). The first sample becomes the main adata object; each following sample is merged with *sc.concat()* and *adata.obs_names_make_unique()* so all four conditions end up in one combined object.

Once all samples are merged, *merge_and_integrate()* handles normalization, highly variable gene selection, scaling, PCA, batch correction (Combat), the neighbor graph, UMAP, and Leiden clustering. Cell types are then assigned with *marker_based_annotation()*, and external CoDi labels are added by *load_codi_annotations()*.

Figures are saved with *save_core_figures()*. Downstream analysis runs in order: *composition_analysis()* compares cell type proportions across conditions; *differential_expression_by_celltype()* returns a DE table used by *pathway_enrichment()* and *size_specific_effects()*. Extra analyses run in *additional_insights()* (cell cycle, IFN signature, antigen presentation, pseudobulk, CoDi vs marker agreement).

Finally, the processed object is written to disk with *adata.write()* as `data/processed/integrated_annotated.h5ad`.

Azimuth annotation is not part of *main()* — it is run separately with *Rscript scripts/azimuth_annotation.R*.

5. Metodology

5.1 Dataset and experimental design

We analysed single-cell RNA sequencing data from human peripheral blood mononuclear cells (PBMC) from one donor exposed to carboxylated polystyrene nanoparticles (PSNPs) in four conditions: 40 nm PSNP, 200 nm PSNP, a 40 nm + 200 nm mixture, and an untreated control. Data were obtained from Zenodo (DOI: 10.5281/zenodo.15866724) as four pre-filtered 10x Genomics .h5ad files. The biological question is whether particle size alters the immune transcriptional response at single-cell resolution.

5.2 Quality control and sample merging

Each sample was loaded separately and filtered using standard QC thresholds: minimum 200 genes per cell, maximum 7,000 genes (doublet filter), minimum 500 UMI counts, and maximum 15% mitochondrial reads. Retention was high across all samples (96.9–98.0%), indicating good input quality. After QC, 8,458 cells (40 nm), 12,421 (200 nm), 6,005 (mix), and 6,356 (control) were retained and merged incrementally on the intersection of genes, yielding a combined dataset of **33,240 cells × 20,388 genes**. These thresholds remove low-quality, stressed, or likely doublet cells while keeping the majority of the data for downstream analysis.

5.3 Integration, dimensionality reduction, and clustering

Merged cells were library-size normalised (10,000 counts per cell), log-transformed, and reduced to the top **3,000 highly variable genes** (Seurat flavour, batch-aware by sample). Expression was scaled, PCA was run (30 components), and **ComBat** batch correction was applied on sample_id to reduce technical differences between the four sample files. A k-nearest-neighbour graph (k = 15) and **UMAP** embedding were computed, followed by **Leiden clustering** (resolution 0.5), which identified **14 clusters**. The UMAP coloured by condition shows that all four exposure groups mix across the embedding after batch correction, meaning cells cluster primarily by cell state/type rather than by sample file.

5.4 Cell type annotation

Immune cell types were assigned using canonical PBMC marker gene sets (T, B, NK, CD14/CD16 monocytes, dendritic cells, platelets). The largest populations were **CD4+ T cells (15,568 cells)**, followed by NK cells, B cells, CD8 cytotoxic T cells, CD14+ monocytes, DCs, CD16+ monocytes, and a small platelet fraction. External **CoDi** reference labels from the dataset were mapped to **99.6%** of cells for validation. Agreement between CoDi (normalised) and marker-based labels was **64.7%**, which is expected given different naming schemes and overlapping cytotoxic/NK marker programmes; it supports that marker annotation is broadly consistent with an independent reference.

5.5 Cell composition across conditions

Composition analysis compared the **fraction** of each cell type within each condition (not raw counts, to account for different sample sizes). **CD4+ T cells** dominated all groups (~44–49%). The overall PBMC profile remained stable across exposures, but notable shifts appeared: **CD14+ monocytes** increased from **5.7%** in control to **9.7%** under 200 nm PSNP, while the mixture condition showed the **lowest** CD14+ monocyte fraction (**3.1%**). These patterns suggest that nanoplastic exposure may subtly reshape immune cell proportions, with the effect depending on particle size and on whether particles are applied alone or as a mix.

5.6 Differential expression

For each major cell type, we ran Wilcoxon rank-sum tests comparing each exposed condition against control (40 nm, 200 nm, and mix separately). **22 comparisons** were completed (platelet DE only for 200 nm vs control due to low cell numbers in other arms). The full table contains **66,000 gene-level results**; **36,814** rows passed significance filters (adjusted $p < 0.05$, $|\log_2FC| > 0.25$). This indicates a **strong transcriptional response** to PSNP exposure within specific immune compartments, even though conditions look mixed on the global UMAP. Monocytes and lymphoid populations both contributed large numbers of differentially expressed genes, especially under 200 nm and mixture exposures.

5.7 Pathway enrichment

Significantly **upregulated** genes from each cell type × comparison were submitted to **Enrichr** (via gseapy) against GO Biological Process, KEGG, and Reactome, producing 89,015

enrichment rows in total. In CD14⁺ monocytes exposed to the mixture, top GO terms included **inflammatory response**, **cytokine-mediated signalling**, **regulation of cell migration**, and **cellular response to lipopolysaccharide** — consistent with innate immune activation. The **40 nm** monocyte response enriched terms related to **MHC class II assembly** and antigen handling, while 200 nm showed enrichment for **cytoplasmic translation** and **platelet activation regulation**, suggesting partly distinct functional programmes by particle size. These results link gene-level changes to interpretable immune and stress pathways.

5.8 Size-specific effects

DE genes were classified by whether they responded specifically to 40 nm, specifically to 200 nm, to both single sizes, to all three exposures, or only to the mixture. In **CD14⁺ monocytes**, **203 genes** were unique to 40 nm vs **81** unique to 200 nm, while **865** responded in all three exposure arms and **294** emerged only in the mixture — supporting the idea that **particle size and combined exposure produce partially distinct transcriptional signatures**. Similar patterns were seen in other lineages (e.g. shared responses across sizes plus mix-only emergent genes in DCs and CD8 T cells). Platelet-specific DE was detected mainly for 200 nm, reflecting sparse platelet numbers in other conditions.

5.9 Additional analyses

An interferon response signature was computed per condition: control showed the highest mean IFN score (0.0033), with exposed groups slightly lower (near zero), indicating **no strong global IFN spike** but room to inspect IFN genes within individual cell types in DE tables. An **antigen presentation score** (HLA-related genes) was lowest in **mix-exposed CD14⁺ monocytes** (−0.77 vs −0.41 in control), hinting at reduced antigen-presentation programme in that condition. A **pseudobulk** count matrix was exported (32 condition × cell-type groups) for optional bulk-style follow-up. **Cell-cycle scores** could not be computed because cell-cycle genes were absent from the post-HVG working matrix — a limitation to note in the report.

5.10 Outputs and pipeline status

The run completed successfully in ~9 minutes. Main outputs include four UMAP figures (umap_condition, umap_clusters, umap_celltypes_marker, composition_barplot), nine CSV tables under results/tables/, and the processed AnnData object data/processed/integrated_annotated.h5ad (33,240 cells × 3,000 HVGs). The optional next

step is **Azimuth PBMC annotation** via Rscript scripts/azimuth_annotation.R, which is required by the project brief but not yet part of this Python pipeline run.

5.11 Summary for presentation (closing paragraph)

In summary, we integrated ~33k PBMC cells from four PSNP exposure conditions, annotated major immune types, and found that nanoplastics induce widespread differential expression across cell types with size- and mixture-specific gene sets. Composition changes were moderate but included a monocyte shift under 200 nm PSNP; pathway analysis pointed to inflammatory and antigen-handling programmes, especially in monocytes. Results are internally consistent (high QC retention, successful batch mixing, CoDi validation ~65%), and the full workflow — including GO/KEGG/Reactome enrichment — completed successfully in the latest pipeline run.

6. Results - what each output file contains and how to read it

Run log

File: results/run_logs/pipeline_run_YYYYMMDD_HHMMSS.txt

This is the full text record of the pipeline execution: configuration, cell counts at each step, which DE comparisons ran, enrichment success/failure messages, and paths to all saved outputs. Use it as a technical appendix — to verify that all steps completed, to quote exact numbers in your report (e.g. “33,240 cells retained”, “36,814 significant DE hits”), and to debug if something fails. It does not contain biological results itself; it documents **how** the results were produced.

Processed data object

File: data/processed/integrated_annotated.h5ad

This is the final **AnnData** object after the full Python pipeline. It contains **~33,240 cells × 3,000 genes** (HVG subset only, not all ~20k genes). Key contents: `adata.X` (normalised/log/scaled expression used for clustering and DE), `adata.obs` (metadata per cell: condition, sample_id, cell_type_marker, cell_type_codi, cluster, QC metrics), `adata.obsm` (UMAP coordinates `X_umap`, `PCA X_pca`), and `adata.layers["counts"]` (raw QC-filtered counts saved before normalisation). Re-open it in Python/Scanpy for extra plots or custom analysis. **Interpretation:** this is your “master object” — all tables and figures were derived from it.

Figure files (results/figures/)

umap_condition.png — Two-dimensional UMAP of all integrated cells, coloured by experimental condition (40 nm, 200 nm, mix, control). **How to read:** each dot is one cell; nearby dots have similar expression. If all four colours appear in the same regions, batch correction worked and samples mix well. Local colour imbalances may suggest condition-specific shifts within a cell state, but UMAP is for **visualisation**, not statistical proof.

umap_clusters.png — Same UMAP, coloured by **Leiden cluster** (14 unsupervised groups). **How to read:** clusters are data-driven groupings without biological names. They show how many transcriptionally distinct populations exist before you assign T cell,

monocyte, etc. Compare with the cell-type UMAP to see whether clusters align with known types.

umap_celltypes_marker.png — UMAP coloured by **marker-based** cell type (cell_type_marker). **How to read:** each region should enrich for a PBMC type (CD4 T, B, NK, monocytes, ...). This validates that marker annotation matches expression structure. Mislabeled regions or mixed colours in one cluster indicate marker overlap or ambiguous cells.

composition_barplot.png — Bar chart of **cell type fractions** per condition (same data as cell_composition_by_condition.csv). **How to read:** bar height = proportion (0–1) of that type within one condition; colours = cell types. Compare bar profiles across the four x-axis groups. Similar profiles → stable composition; taller monocyte or shorter T cell bars under PSNP → possible exposure-driven shift in immune makeup.

Table files (results/tables/)

cell_composition_by_condition.csv (32 rows)

Columns: **condition** (PSNP_40nm, PSNP_200nm, PSNP_mix_40_200, control), **cell_type_marker**, **n_cells** (absolute count), **fraction** (share within that condition, sums to 1.0 per condition). **How to interpret:** use **fraction** for fair comparison across samples of different size. Example from your run: CD14+ monocytes rise from **5.7%** (control) to **9.7%** (200 nm) but drop to **3.1%** (mix). CD4+ T cells stay ~44–49% everywhere. This table supports the composition section of the assignment; pair it with the barplot in slides.

differential_expression_all.csv (~66,000 rows)

Columns: **names** (gene), **logfoldchanges** (log fold change, exposed vs control), **pvals**, **pvals_adj** (FDR-adjusted p-value), **scores** (Wilcoxon statistic), **cell_type**, **comparison** (e.g. PSNP_40nm_vs_control). **How to interpret:** filter rows where **pvals_adj** < **0.05** and **|logfoldchanges|** > **0.25** for “significant” genes. Positive logFC → higher in exposed; negative → higher in control. Always filter by **cell_type** and **comparison** together — each row is one gene in one cell type for one exposure test. Your run had **36,814** significant rows total. Use this table to list top genes in slides (e.g. monocyte + 40 nm) and as input logic for size-specific and pathway steps.

pathway_enrichment_all.csv (~89,015 rows)

Columns include **Term** (pathway/process name), **Adjusted P-value**, **Odds Ratio**, **Genes** (input genes hitting the term), **Overlap**,

plus **cell_type**, **comparison**, **gene_set** (GO / KEGG / Reactome). **How to interpret:** each row = one enriched pathway for one cell type and one exposure vs control, using **upregulated** DE genes only. Lower **Adjusted P-value** = stronger enrichment. Sort by adjusted p-value within a group (e.g. Monocyte_CD14 + mix) to find themes — your mix monocyte run enriched **inflammatory response, cytokine-mediated signalling, response to LPS**. Use **Genes** to see which DE genes drove the term. This table answers “what biological processes are affected?” for the assignment.

size_specific_effects_summary.csv (40 rows)

Columns: **cell_type**, **effect_class**, **n_genes**. **effect_class** values: unique_40nm, unique_200nm, shared_40_200, shared_all_three, mix_only_emergent. **How to interpret:** counts of significant DE genes ($p_{adj} < 0.05$, $|\log FC| > 0.25$) classified by which exposure(s) they respond to. Example: Monocyte_CD14 — **203** genes only for 40 nm, **81** only for 200 nm, **865** for all three exposures, **294** only for mix. This directly supports the assignment’s **size-specific effects** section: smaller particles, larger particles, shared PSNP response, and mixture-only emergent response.

annotation_agreement_metrics.csv (1 row)

Columns: **metric** (codi_marker_agreement), **value** (**0.647** in your run). **How to interpret:** fraction of cells where normalised CoDi label equals cell_type_marker (0–1, or 64.7%). ~65% means reasonable agreement with an external reference, not perfect — useful for validation slides. It does not mean 35% of cells are “wrong”; many mismatches come from different label granularity (e.g. CoDi “CD4+ T cell” vs pipeline CD4_T) or overlapping marker scores.

ifn_scores_by_condition.csv (4 rows)

Columns: **condition**, **IFN_score** (mean interferon signature across all cells in that condition). **How to interpret:** relative innate immune / interferon activation. Your values are small and near zero; control (**0.0033**) is slightly highest. Conclusion: **no strong global IFN activation** by PSNP in this summary — but check IFN-related genes in DE for specific cell types if needed.

antigen_presentation_scores.csv (32 rows)

Columns: **condition**, **cell_type_marker**, **antigen_presentation_score** (mean HLA-related gene score). **How to interpret:** higher score → stronger antigen-presentation programme in that type under that condition. B cells score high (~1.5–2.0) as expected. For **Monocyte_CD14**, mix exposure shows the **lowest** score (**–0.77** vs **–0.41** control),

suggesting reduced antigen-presentation signature in mix-treated monocytes — a candidate biological talking point when combined with pathway/DE results.

cell_cycle_scores_by_condition.csv (4 rows, empty scores in your run)

Columns: **condition**, **S_score**, **G2M_score**. **How to interpret:** mean cell-cycle phase scores (proliferation proxy). In your run both columns are **NaN** because cell-cycle genes were not in the 3,000 HVG matrix. **Do not use this file in conclusions** unless you re-run with full genes or add cycle genes to the panel. Mention as a limitation.

pseudobulk_counts_condition_celltype.csv (32 rows × ~3,000 gene columns)

Each row = one **condition** × **cell_type** group; columns = genes with **summed raw UMI counts** across all cells in that group. **How to interpret:** a “bulk-like” matrix for optional downstream tools (DESeq2, edgeR, etc.). Not used by the main pipeline plots. High counts in a column for one group vs another hint at cell-type-specific expression differences at the pseudobulk level.

azimuth_annotations.csv (optional, not yet from your Python run)

Produced by **Rscript scripts/azimuth_annotation.R**. Expected columns include Azimuth-predicted PBMC labels and confidence scores per cell. **How to interpret:** reference-atlas annotation — compare with cell_type_marker and CoDi for the assignment’s “Azimuth PBMC reference” requirement. Run this separately if not done yet.

How files relate to each other (one paragraph)

The pipeline flows from **integrated_annotated.h5ad** (single-cell object) to **UMAP figures** (visual overview) and **cell_composition_by_condition.csv** (population shifts). **differential_expression_all.csv** tests gene changes per cell type and exposure; **pathway_enrichment_all.csv** summarises those gene lists into GO/KEGG/Reactome terms; **size_specific_effects_summary.csv** classifies DE genes by particle size. Signature tables (**IFN**, **antigen presentation**) add interpretable immune scores; **annotation_agreement_metrics.csv** validates marker labels against CoDi. The **run log** documents the run; use all together in the report: figures for overview, composition for “who changed”, DE + pathways + size-specific for “what changed and how it depends on particle size”.

Optional slide captions (one line per file)

- **Log:** Complete audit trail of pipeline steps and cell counts.
- **h5ad:** Processed single-cell object for re-analysis.
- **UMAP condition:** Batch-corrected mixing and global exposure patterns.
- **UMAP clusters / cell types:** Unsupervised vs annotated structure.
- **Composition CSV/barplot:** Cell type proportions per PSNP condition.
- **DE CSV:** Gene-level exposure vs control tests per cell type.
- **Pathway CSV:** Enriched biological processes from upregulated genes.
- **Size-specific CSV:** Genes unique to 40 nm, 200 nm, shared, or mix-only.
- **Agreement CSV:** CoDi vs marker label concordance (~65%).
- **IFN / antigen scores:** Immune signature summaries by condition and type.